

Unit1 概述

软件架构的思想和特征

- 软件架构的主要思想是将注意力集中在系统总体结构的组织上。
- 软件架构的特征（重利分质概循）

注重可重用性	组件及架构级重用	提高软件质量
利益相关者较多	满足各利益相关者需求	平衡需求
关注点分离	分而治之、模块化	简化复杂性
质量驱动	使用软件架构来处理质量属性需求、控制复杂性	由功能、数据流驱动向质量驱动转变
概念完整性	强调设计决策是一个持续的过程	每个决策都要在其前面设计决策的基础上进行
循环风格	架构风格、架构模式	用标准方法来处理反复出现的问题

Unit2 软件架构定义

组成派定义

关注于**软件本身**，将软件架构看做**组件和交互的集合**，认为软件架构是软件设计过程的**层次之一**，该层次**超越**计算过程中的算法设计和数据结构设计。 软件架构包括组件（component）、连接件（connector）和约束（constraint）三大要素。

组件可以是一组代码，也可以是独立的程序；

连接件可以是过程调用、管道和消息等，用于表示组件之间的相互关系；

约束一般为组件连接时的条件。

决策派定义

关注于软件架构中的**实体（人）**，将软件架构视为一系列**重要设计决策的集合**。
集合包括：

软件系统的组织；

组成系统的结构元素和它们之间的接口，以及当这些元素相互协作时所体现的行为；

如何组合这些元素，使它们逐渐合成为更大的子系统；
架构风格；
这些元素以及它们的接口、协作和组合

参考定义框架

软件架构一般由如下五种元素构成：**组件**（component）、**连接件**（connector）、**配置**（configuration）、**端口**（port）和**角色**（role）。

组件：具有某种功能的可重用的软件模块单元，表示了系统中主要的计算单元和数据存储。

连接件：表示了组件之间的交互。

配置：表示了组件和连接件的拓扑逻辑和约束（constraint）。

端口：组件作为一个封装的实体，只能通过其接口与外部交互，组件的接口由一组端口组成，每个端口表示了组件和外部环境的交汇点。

角色：连接件作为建模软件体系结构的主要实体，同样也有接口，连接件的接口由一组角色组成，连接件的每个角色定义了该连接件表示的交互的参与者。

Unit 3 软件架构模型

软件架构建模的五类方法（过程、优缺点）

基于 UML 的建模方法（画图）

其他建模方法（文本语言、MDA）

“4+1”模型（分别对应哪些 UML 图）

。基于非规范的图形表示的建模方法

过程：将软件架构按照图形的方式进行表达。

优点：便于理解，建模简单

缺点：不够规范，产生歧义

。基于 UML 的建模方法

过程：将 UML 看作是一种软件架构描述语言直接对架构建模；

扩展机制约束 UML 的元模型；

对 UML 的元模型进行扩充

优点：统一的表示方法、支持多视图、有效利用模型操作工具；统一的交叉引用（cross-referencing）模型信息的方法，有利于维护开发元素的可处理性，避免错误的产生。

缺点：架构的支持性不强，语义不够精准，信息冗余，非形式化的建模层次不够

。基于形式化的建模方法

过程：利用一些已知特性的数学抽象来为目标软件系统的状态特征和行为特征构造模型

优点：保证其书写的规格说明文档的正确性，同时还能保证有很好的可读性和可理解性

缺点：建模复杂

。基于 UML 形式化的方法

过程：利用形式化与 UML 结合的建模方法研究成果，对 UML 图形赋予形式化语义，然后就可以利用已有的形式化语言和工具对 UML 模型进行推理验证。

优点：对 UML 图形赋予形式化语义，然后就可以利用已有的形式化语言和工具对 UML 模型进行推理验证

缺点：UML 中的类图、用例图、状态图以及顺序图较适合形式化描述，其他的图用形式化的描述方法反而使得描述过程变得更加复杂，容易降低效率

。其他建模方法

■文本语言建模方法

文本语言建模是通过文本文件（通常需要符合某些特殊的句法格式）描绘架构

优点：

- 单文档可描述整体架构，便于文本编辑器与用户交互；
- 多种工具支持生成库文件进行句法分析与检查；
- 编辑器通常附带开发支持工具。

缺点：

- 表示类图形结构不直观；
- 编辑器多限于满屏连续文本，难以以其他形式组织。

■模型驱动的架构建模方法（MDA）

MDA 致力于将软件开发从以代码为中心提高到以模型为中心，使模型不仅被作为设计文档和规格说明来使用，更成为一种能够自动转换为最终可运行系统的重要软件制品

在 MDA 中，将模型区分为 PIM（平台无关模型）和 PSM（平台相关模型）
核心思想是抽象出与**实现技术无关、完整描述业务功能的平台独立模型**。

优点：

将模型与实现分离后，能够很好的**适应技术易变性**。

由于实现往往高度依赖特定技术和特定平台，当技术发生迁移时，只需针对这种技术作相应的实现，编写相应的运行平台或变换工具。所以，能够比较好的应对实现技术发展带来的挑战。

缺点：三种模型理解不一致；上层模型下层不好实现

“4+1” 模型（用逻辑开物）

用例视图（场景）：从外部世界的角度描述正在建模系统的功能

用例图，描述和概述图

逻辑视图：描述系统的组成方式和交互方式

类图、对象图、状态图和协作图

过程视图：描述系统的进程

活动图、顺序图

开发视图：系统的模块、组件管理

包图和组件图

物理视图：软件如何映射到现实世界

部署图

Unit 4 软件架构风格与模式

什么是软件架构风格

软件架构风格是描述某一特定应用领域中系统组织方式的**惯用模式**，作为“可复用的组织模式和习语”，为设计人员的交流提供了公共的术语空间，促进了设计复用与代码复用。

使用架构风格的好处（重组互析）

好处：

- 促进设计的重用性和代码的重用性
- 系统的组织结构易被理解
- 较好地支持系统内部的互操作性
- 便于针对特定风格的分析

20 种风格特点、优点、缺点、适用范围

数据流风格（包含管道过滤器风格和批处理风格）

特点：

- 数据可用性决定处理执行；
- 系统结构取决于数据在处理间的有序流动；
- 纯数据流系统中，处理间仅进行数据交换，无其他交互。

优点：

各个组件相互独立、支持功能模块的复用、具有较强的可维护性和可扩展性、支持一些特定的分析、具有并发性

缺点：

大量处理过程、数据流进行解析或反解析复杂、交互弱

适用范围：

数据源源不断地产生，系统需要对这些数据进行若干处理

批处理风格

特点：近乎线性；每一个处理步骤都是独立的应用程序；每一步在上一步结束之后发生；传出数据必须完整，以整体形式传送

适用范围：

数据一批一批地产生，系统需要对这些数据进行若干处理

管道过滤器：

优点：（**独复维特并**）各个组件相互独立、支持功能模块的复用、具有较强的可维护性和可扩展性、支持一些特定的分析、具有并发性

不足：（**大数交**）大量处理过程、数据流进行解析或反解析复杂、交互弱

适用范围：

数据源源不断地产生，系统需要对这些数据进行若干处理

主程序子程序风格

特点：

- 从功能视角设计系统，通过逐步分解细化形成架构；
- 将大系统模块化，主程序调用模块实现系统功能；
- 主程序正确性依赖子程序正确性。

优点：

结构清晰、开发过程逐步细化

缺点：（**存规复**）

数据存储格式要求一致、规模不能太大、难以复用

适用范围：

它适用于可以通过过程定义层次结构适当定义计算的应用

面向对象风格

特点：

1. 对象通过保持表示上的不变式维护其完整性；
2. 对象的表示对其他对象是隐蔽的，抽象数据类型和面向对象系统已被广泛使用。

优点：（**改完存**）

对象易于修改、完全性高、数据存取改为交互的程序集合

缺点：（**象交责捕**）

对象太多、交互太多、行为责任分散、捕获相关设计族

适用范围：

适用于中心问题是识别和保护相关信息体（尤其是表示信息）的应用

层次化风格

特点：

问题分解成渐进、不支持跨层交互、修改一层通常只会影响上层、上层必须了解下层

优点：（解层重）

复杂问题分解成一个增量步骤序列、层次影响小、支持层的重用（扩展）

缺点：（适模性）

适用性窄、建模困难、性能受限

适用范围：

适用于涉及可分层排列的不同服务类别的应用程序

客户机/服务器风格

特点：

两个相互独立的逻辑系统，为了完成特定的任务而形成一种协作关系，资源对等

两层：

优点：（分透异灵开）

C 与 S 分开，灵活性较高：数据、软件、硬件、整体成本较低

- （1）客户机组件和服务器组件分别运行在不同的计算机上，有利于分布式数据的组织和处理。
- （2）组件之间的位置是相互透明的
- （3）客户机程序和服务器程序可运行在不同的操作系统上，便于实现异构环境和多种不同开发技术的融合。
- （4）软件环境和硬件环境的配置具有极大的灵活性，易于系统功能的扩展。
- （5）将大规模的业务逻辑分布到多个通过网络连接的低成本的计算机上，降低了系统的整体开销

缺点：（成专单扩安维）

S 的成本高、需要专门程序、传输形式单一、难以扩展到网络、数据安全性低、维护成本高

三层：

优点：（逻辑环并安）

系统的逻辑结构清晰、可维护性强、环境选择性高、开发并行度高、安全性好

缺点：

通信效率问题、独立性要求

适用范围：数据和处理分布在一定范围内的各个组件上，组件之间通过网络连接

浏览器/服务器风格

特点：

客户端仅需浏览器，轻量化；
Web 服务器负责核心功能；
维护集中于服务器端，管理便捷；

属于三层 C/S 结构的实现。

优点：

客户端操作简单、通信协议统一、开发成本低

缺点：（个负交扩安速）

个性化程度低、S 的负载重、交互性较弱、可扩展性差、安全性不足、速度较慢

适用范围：

分布式应用与在线办公；

需频繁更新的系统；

跨平台与低成本场景。

仓库风格

特点：

仓库是存储和维护数据的中心场所

优点：

便于模块间的数据共享、方便模块的添加、更新和删除；避免了知识源的不必要的重复存储

缺点：

需要一定的同步/加锁机制保证数据结构的完整性和一致性

适用范围：

应用于核心问题是建立、扩充和维护一个复杂的中央信息体的情况；

传统数据库构建业务决策系统；构建软件开发环境

黑板系统风格

特点：

没有直接的算法，多种方法都可能解决问题，问题没有唯一的解答，需要多领域知识协作解决

优点：

多客户共享大量数据、易于扩展、知识源可重用、容错性

缺点：

同步、数据结构要一致、数据结构较难更改

适用范围：

一个大问题被分解为若干个子问题，每个子问题的解决需要不同的问题表达方式和求解模型，分别设计求解程序；起源与人工智能领域，语音处理等

解释器风格

特点：

执行其他程序的程序，针对不同的硬件平台实现了一个虚拟机，将高抽象层次的程序翻译为低抽象层次所能理解的指令

优点：

提升可移植性、便于模拟测试

缺点：

性能下降

适用范围：JIT；应用程序不能直接运行在最合适的机器上。不能直接以最合适的语言执行

基于规则的系统风格

特点：

把频繁变化的业务逻辑抽取出来，形成独立的规则库。

业务逻辑=固定业务逻辑+可变业务逻辑(规则)+规则引擎

优点：

降低了修改业务逻辑的成本、缩短了开发时间、规则共享、规则改变迅速风险低

事件驱动风格

特点：

组件广播事件，其他组件通过监听事件产生隐式调用

事件的触发者并不知道哪些构件会被这些事件影响，相互保持独立。

隐式调用：事件驱动风格是隐式调用，但是事件驱动系统可以使用显示方法调用

优点：

组件相对独立、组件复用性强、便于维护

缺点：

组件的掌控性弱、数据交换格式、正确性验证问题

应用范围：

断点、win32GUI

C2 风格

特点：

基于组件和消息层次体系架构风格

优点：

语言自由、组件易换、支持多用户多系统、组件不需共享地址空间、动态更新系统框架结构

缺点：

不太适合大规模流式风格系统，不适合数据库访问频繁

适用范围：

适用于 GUI 软件开发，构建灵活和可扩展的应用系统

平台/插件风格

特点：

不修改程序主体（或者程序运行平台）的情况下对软件功能进行扩展与加强

优点：

降低模块依赖性、各模块独立开发、系统可拆装

缺点:

插件在不同平台可重用性差

面向 Agent 风格

特点:

事物属性受环境影响。将事物以及影响因素结合为 Agent 作为系统的基本单位。

优点:

便于解决复杂问题，分布开放异构的软件环境

缺点:

大多数结构中 Agent 缺乏社会型结构描述与环境交互复杂性

面向方面软件架构风格

特点:

增加了方面组件来封装系统的横切关注点

优点:

关注点分离、与面向对象互补、对横切关注点模块化

缺点:

不好定义；不好实现

面向服务架构风格

特点:

SOA 是一个组件模型，将服务通过接口联系起来。

优点:

灵活、松耦合、易维护、粗粒度、对 IT 资产的复用、使企业的信息化建设真正以业务为核心

缺点:

服务划分困难、接口标准不统一

- (1) 服务的划分很困难。
- (2) 服务的编排是否得当。
- (3) 如果选择的接口标准有问题，如主流的 Web service 之类，会带来系统的额外开销和不稳定性。
- (4) 对 IT 硬件资产还谈不上复用。
- (5) 目前主流实现方式接口很多，很难统一。
- (6) 目前主流实现方式只局限于不带界面的服务的共享。

微服务架构风格

特点:

将单个应用程序开发为一组小型服务，每个服务独立运行，通过网络通信

优点：

各个模块独立部署、松耦合、细粒度的扩容

缺点：

分布后的复杂性、接口统一、代码重复、异步问题、维护麻烦、其他的像不可变基础设施、管理难度、微服务架构的推进等。

正交架构风格

特点：

线索之间是相互独立的(正交的)；应用系统垂直分割为若干个线索(子系统)，线索又分为几个层次（系统有一个公共驱动层(一般为最高层)和公共数据结构(一般为最低层)）

优点：

结构清晰、易修改、可移植性好

缺点：

正交化的成本、适用性较低

异构风格

特点：

异构架构是几种风格的组合

优点：

遗留代码重用、不同标准的组合

缺点：

风格的兼容性

基于层次消息总线的架构风格

特点：

各个组件挂接在消息总线上，向总线登记感兴趣的消息类型

优点：

降低耦合、增强组件重用、易维护、支持系统演化

缺点：

系统重用要求高，可重用性差

应用场景：

青鸟

模型-视图-控制器风格

特点：

MVC 结构主要包括模型、视图和控制器三部分

优点：

多视图与一模型对应，同步性好、具有一定的移植性、易修改

缺点：

系统设计复杂、视图与控制器重用性低、访问效率低（需要视图访问多个接口）、频繁访问未变化的数据，也将降低系统的性能

Unit 6 软件架构与敏捷开发

敏捷开发如何改变了软件架构的设计方式

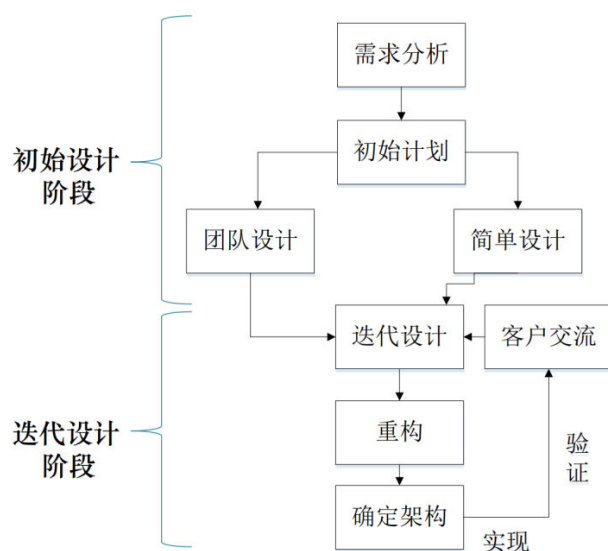
- 敏捷开发非常重视软件的架构设计，但是轻架构的详细设计。
- 敏捷思想中将传统的架构设计分成：种子架构设计+详细架构设计。
- 种子架构设计关注软件系统的骨架或轮廓的设计。
- 敏捷开发将详细架构设计转移到 Code 编码阶段、重构阶段、单元测试阶段等。
- 分离后，敏捷软件种子架构的内容包括：软件的架构层次，重要模块、重要类的说明（无须设计全部的类和方法）等。
- 敏捷开发把传统软件开发前期的详细架构设计，分散到了整个敏捷开发软件过程中，以达到提高效率、减少风险的目的。

两类常见敏捷软件架构设计方法

优秀的敏捷软件架构的设计过程一般同时包含规划式设计和演进式设计，具体体现为初始阶段设计和迭代过程中的设计。

（规划式设计）初始设计：确定系统的基本流程、结构、划分，得到一个原始架构。

（演进式设计）迭代过程：相对稳定，每次基于上一次迭代的结果。使架构越来越强壮。



敏捷软件架构的一般设计过程

Unit 8 软件架构建模方法

成功的软件架构应具有的品质（模变运数部）

- 良好的模块化
- 适应功能需求的变化，适应技术的变化
- 对系统的动态运行有良好的规划
- 对数据的良好规划
- 明确、灵活的部署规划

将软件架构映射到详细设计经常遇到什么问题？如何解决？（视统交设）

- 缺失重要架构视图，片面强调功能需求。
 针对遗漏的架构视图进行设计
- 不够深入，架构设计方案过于笼统，基本还停留在概念性架构的层面，没有提供明确的技术蓝图
 将设计决策细化到和技术相关的层面。
- 缺失层次之间的交互接口和交互机制，只进行职责划分
 步步深入，明确各层之间的交互接口和交互机制
- 过度设计
 切忌过度设计

MDA 的基本思想、过程，应用 MDA 的好处

MDA 的根本思想

将软件系统分成**模型和实现**两部分：模型是对系统的描述，实现是利用特定技术在特定平台或环境中对模型的解释。模型仅仅负责对系统的描述，与实现技术无关。这是模型的实现技术无关性。

MDA 的过程

- （1）用计算无关模型 CIM **捕获需求**；
- （2）**创建**平台无关模型 PIM；
- （3）将 PIM **转化**成为一个或多个平台特定模型 PSM，并加入平台特定的规则和代码；
- （4）将 PSM 转化为**代码**等。

MDA 的好处

将模型与实现分离后，能够很好的**适应技术易变性**。

由于实现往往高度依赖特定技术和特定平台，当技术发生迁移时，只需针对这种技术作相应的实现，编写相应的运行平台或变换工具。所以，能够比较好的应对实现技术发展带来的挑战。

架构设计面临的主要威胁及对策（非变全及创执）

- (1) 被忽略的重要非功能需求
 全面认识需求
- (2) 频繁变化的需求
 关键需求决定架构
- (3) 考虑不全面的架构设计
 多视图探寻架构
- (4) 不及时的架构验证
 尽早验证架构
- (5) 较高的创造性的架构比重
 合理分配经验架构与创新架构比重
- (6) 架构的低可执行性
 验证架构的可执行性

Unit 9-10 软件架构质量与测试

软件架构评估的方式分类

	基于问卷调查或检查表的评估技术	基于场景的评估技术	基于度量的评估技术
通用性	通用或特定领域	特定领域	通用或特定领域
评估者对待评估架构的掌握程度要求	简单了解	比较熟悉	精确掌握
客观性	主观	比较主观	比较客观
评估时间	早期,中期	中期	后期

质量属性、（质量）场景

（1） 内部质量指标

可维护性（纠正错误、提升性能、适应环境）

可重用性（组件重用）

可移植性（不同计算平台运行）

可集成性（组件协同工作）

可测试性（测试发现缺陷）

（2） 外部质量指标

性能（响应能力）

可用性（系统正常运行的时间比例）

可靠性（容错、健壮性）

安全性（阻止非授权使用、拒绝服务攻击、阻止信息泄露丢失）

易用性（可学习性、效率、错误处理、错误避免……）

（质量）场景

场景：采用刺激、环境和响应三方面来对场景进行描述

刺激源：产生刺激的实体

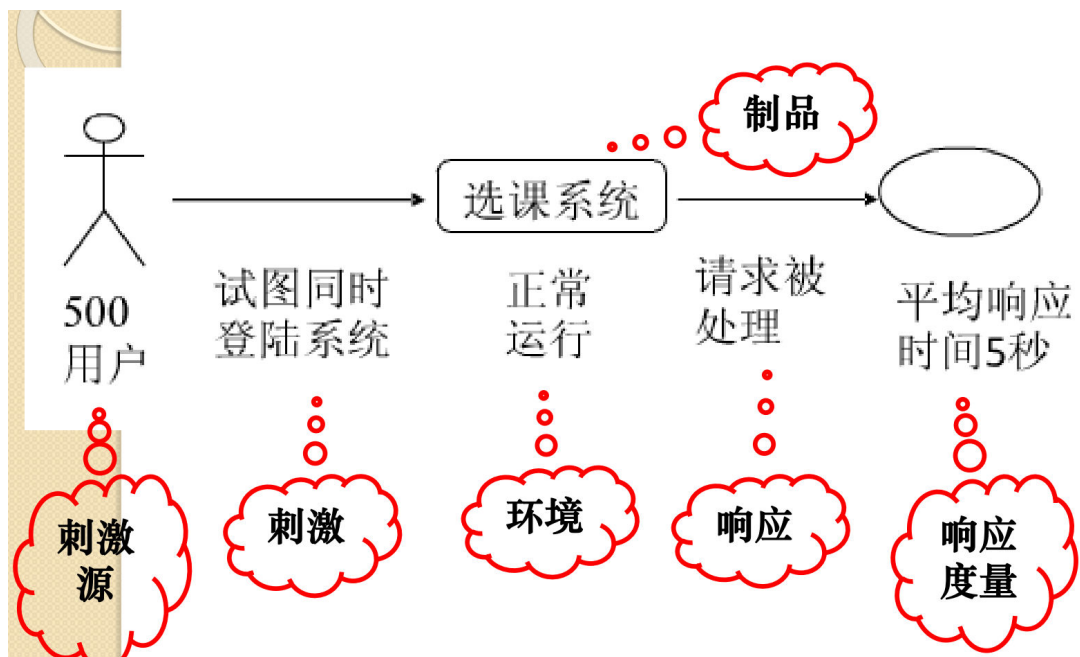
刺激：风险承担者的动作

环境：系统的状况

制品：当前系统

响应：环境对刺激产生的结果

响应度量：对响应的度量



ATAM

敏感点是一个或多个质量属性的特征

权衡点是影响多个质量属性的特征，多个质量属性的敏感点，权衡点影响多个质量属性

质量属性效用树说明构成系统“效用”的质量属性（性能、有效性、安全性、可修改性、可用性），具体到场景层次，标注刺激/反应，并区分不同的优先级

质量属性效用树的构建

评估小组与项目决策者合作，共同确定出该系统的最重要的质量属性目标，并设置优先级，进行进一步的细化，该步指导其它步骤的分析

ATAM 优缺点

优点：考虑到了所有与系统相关的人员对质量的要求；

- 。确定了功能和结构之间的映射，设计体现待评估质量属性的场景，分析软件体系结构对场景的支持程度；从多个竞争的质量属性方面来理解

缺点：是特定于领域的；需要有丰富的领域知识；必须对待评估的软件体系结构有一定的了解

SAAM 优缺点

SAAM 考查的是 SA 单独的质量属性，而 ATAM 提供从多个竞争的质量属性方面来理解 SA，使用户能认识到在多个质量属性目标间权衡的必要性

Unit 11-12 软件架构度量与架构演化

为什么要进行软件架构度量？（效早演静）

- 。 （1）从架构出发，可以更有效把握整个软件系统的设计结构和版本更新问题。
- 。 （2）基于架构的度量可以在设计早期发现软件架构的不合理之处。
- 。 （3）基于架构的度量可以对架构的演化程度进行评估和预测。
- 。 （4）架构度量是一种静态的度量方法，不需要执行软件或者进行仿真，具有成本低、容易实现且不依赖于程序特定的运行环境的优点。

架构度量和架构演化的关系。

相辅相成；

将基于度量的软件架构评估方法应用于软件演化，通过对演化前后不同版本的架构进行度量，分析相关质量属性在每次演化之后的变化情况，找出架构内部结构的变化与外部质量属性变化间的关联。

软件架构为什么会发生演化？

软件架构的演化和维护是一个不断迭代的过程，通过演化和维护，软件架构逐步得到完善，以满足用户需求

软件架构演化的方式

针对软件架构的演化过程是否处于系统运行时期，可以将软件架构演化分为软件架构静态演化和软件架构动态演化。

。 软件架构静态演化

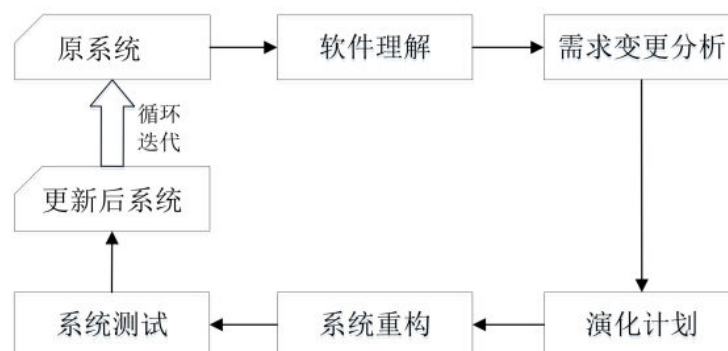
定义： 软件静态演化是指系统**停止运行期间的修改和更新**，即一般意义上的修复和升级

必要性：

设计时演化需求： 在架构开发和实现过程中**对原有架构进行调整**，保证软件实现与架构的一致性以及软件开发过程的顺利进行；

运行前演化需求： 软件发布之后**由于运行环境的变化**，需要对软件进行修改升级，在此期间软件的架构同样要进行演化。

演化过程：



。 软件架构动态演化

定义： 动态演化是在系统运行期间的演化，需要在不停止系统功能的情况下完成演化。发生在有限制的运行时演化和运行时演化阶段。

必要性：

软件内部执行所导致的体系结构改变

是软件系统外部的请求对软件进行的重配置

内容：

。 **属性改名：** 在运行过程中，用户可能会对非功能指标进行重新定义，如服务响应时间等。

。 **行为变化：** 在运行过程中，用户需求变化或系统自身服务质量的调节，都将引发软件行为的变化。如：为了提高安全级别而更换加密算法；将 http 协议改为 https 协议。

。 **拓扑结构改变：** 如增、删组件，增、删连接件，改变组件与连接件之间的关联关系等。

。 **风格变化：** 一般软件演化后其架构风格应当保持不变，如果非要改变

软件的架构风格，也只能将架构风格变为其“衍生”风格，如将两层 C/S 结构调整为三层 C/S 结构或 C/S 和 B/S 的混合结构

方式：

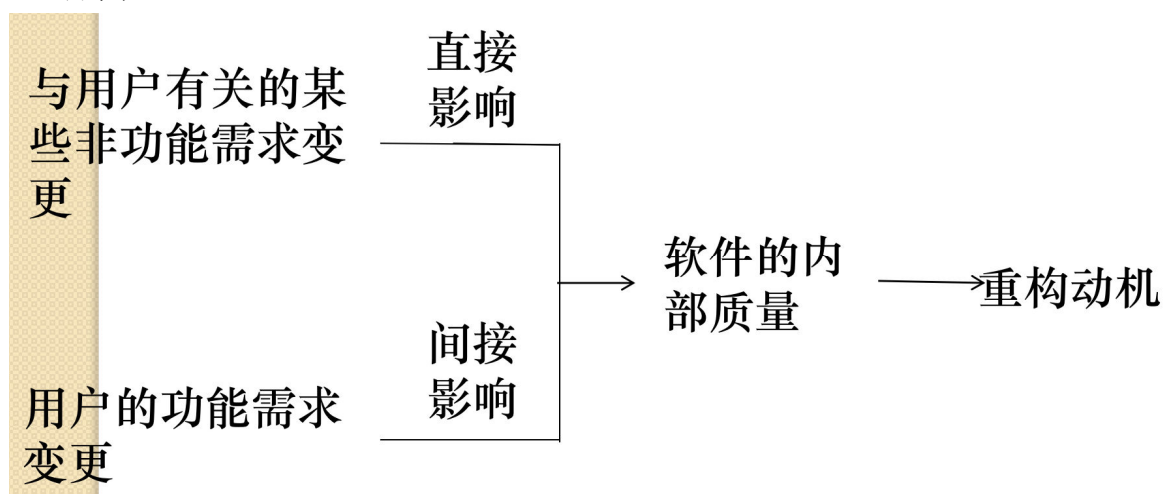
采用动态软件架构

进行动态重配置

Unit 13 软件架构重构

什么是软件架构重构？为什么要进行软件架构重构？

- 软件架构重构是在不改变软件的功能和外部可见性的情况下，为了改善软件的结构，提高软件的可读性、可扩展性、可重用性、可维护性等质量属性的情况下而对其进行的改造。
- 主要过程是重构点识别与对重构点实施重构操作。
- 软件重构是进行软件优化和提高软件质量的重要手段之一。
- 软件重构就是改进已经写好的软件设计、提高软件的某些质量属性。
- 原因：



软件重构的类别：架构重构、结构调整、代码重构

重构类别	重构需求	重构目的
架构重构	软件架构级坏味道、设计技术债	通过软件产品整体结构或者组织方式的重构来消除坏味道和技术债，提高软件架构的质量等
结构调整		优化模块内部的组织和依赖关系，提高模块的可维护性和可扩展性
代码重构	代码坏味道、代码技术债	通过代码重构来消除坏味道和代码方面的技术债，提高代码质量等

Unit 14-15 架构腐蚀，架构恢复

架构腐蚀的含义

预期软件架构或概念软件架构与实际软件架构之间的偏离。这种偏离更多的是源自日常的软件修改，而非人为的恶意。

架构恢复的含义及意义

含义：从工程项目中获取所需的架构信息，恢复出架构的组成元素。

意义：从源代码或其他架构文档中抽取架构，为腐蚀的检测、评价和修复奠定基础。

架构恢复与架构腐蚀的关系

- 相辅相成：
 - 架构恢复是应对架构腐蚀的重要工具，通过从源码或文档中提取信息重建架构，帮助检测和理解腐蚀。
 - 没有架构恢复的支持，修复腐蚀（尤其是复杂系统中的腐蚀）可能无法实现。
- 共同目标：
 - 两者的共同目标是确保架构与实现的一致性，提升系统的可维护性和质量。
 - 架构恢复为架构腐蚀修补提供数据支持，而架构腐蚀修补策略推动了架构恢复技术的发展

Unit 16-18 技术债、坏味道、脆弱性

软件技术债的含义及分类

含义：开发人员为了加速软件开发（或修改），或是由于自身经验的缺乏，有意或无意地在应该采用最佳方案时进行了妥协，使用了短期内能加速软件开发的方案，从而在未来给自己带来的额外开发负担。

分类：（代设测文）

- 代码债（开发时没有守代码规范导致的债务）编码风格不一致
- 设计债（在设计时未采用最优解决方案导致的债务）违背设计规则
- 测试债（测试环节产生的债务）测试覆盖面不充分
- 文档债（由技术文档问题产生的债务）缺文档

技术债的成因（进经重设有）

- 进度压力
- 软件设计师缺乏足够的经验和技巧
- 不注重设计原则的应用
- 缺乏对设计坏味和重构的意识
- 开发中有意采用非最优的选择

技术债的偿还

- (1) 发现项目中包含的技术债；
- (2) 将技术债加入到产品列表中；
- (3) 根据债务的影响和偿还成本进行优先级排序；
- (4) 在合适的开发周期中对不同技术债进行修改偿还。

软件坏味道的含义

软件中的一些潜在问题或不良设计痕迹，这些问题可能不会直接导致软件错误，但会在一定程度上影响软件的可维护性、可扩展性和可理解性。

代码坏味（重复代码）

如果程序中的某一段代码是不稳定或者有一些潜在问题的，那么该段代码往往会包含一些明显的不太好的痕迹。我们称这些痕迹为“代码坏味或者代码异味”。

架构坏味（未使用的接口）

一种通常使用的、可以对系统生命周期特性产生消极影响的架构设计。

软件脆弱性的含义

软件脆弱性是导致破坏系统安全策略的系统安全规范、系统设计、实现和内部控制等方面的弱点。

软件架构脆弱性的成因

软件（系统）架构设计存在一些明显的或者隐含的缺陷，攻击者可以利用这些缺陷攻击系统，或者当受到某个或某些外部刺激时，系统发生性能下降、稳定性下降、可靠性下降、安全性下降等等。如果软件架构具备这类缺陷，我们认为该软件架构是脆弱的，也就是软件架构脆弱性。

软件架构的风格（style）和模式（pattern）有关（分层架构等）